ATTENDEE PREPARATION GUIDE

Agentic Mobile QA — before you arrive

A 3-hour hands-on workshop. Everything runs locally on your machine: an Android emulator, Appium, and an LLM served by Ollama.

⏳ DO THIS 4–5 DAYS AHEAD · 🔑 NO API KEY NEEDED · 💻 macOS · LINUX · WINDOWS + WSL2

Please finish this before the session — and verify it.

The setup is heavy: several GB of downloads plus an Android emulator. The 25-minute setup block on the day is for **fixing stragglers, not first-time installs**. If you arrive without a verified checklist, you will not be able to follow the hands-on labs. Start on good Wi-Fi, well ahead of time.

1 💻 Hardware & accounts

- ☐ **macOS** (Apple Silicon or Intel), **Linux**, or **Windows + WSL2**
- ☐ **≥ 16 GB RAM** — the emulator and the 8B model both run locally. 8 GB will struggle.
- ☐ **~20 GB free disk** — Android system image ~4 GB, emulator ~1 GB, JDK/SDK ~2 GB, llama3.1 ~4.9 GB, plus node_modules and a ~120 MB demo APK.
- ☐ A **GitHub account** — read access is enough (used in the CI chapter).

2 ⚙️ Toolchain

Install each one, then run its verify command. Every command must succeed.

TOOL	INSTALL	VERIFY
Node.js 20+	nodejs.org	`node -v`
pnpm 10	`corepack enable` (ships with Node)	`pnpm -v`
JDK 17	macOS: `brew install openjdk@17` · else adoptium.net	`java -version`
Android SDK	Android Studio, **or** command-line tools (§3)	`adb --version`
Build-tools, emulator, platform 34, system image	via `sdkmanager` (§3)	`emulator -version`
Appium 3	`npm i -g appium`	`appium --version`
UiAutomator2 driver	`appium driver install uiautomator2`	`appium driver list --installed`
Ollama	ollama.com	`ollama --version`
AppClaw CLI	`npm i -g @appclaw/cli`	`appclaw --version`

> ⚠️ **Two traps that cost people an evening**
>
> **macOS JDK:** do **not** use `brew install --cask temurin` — it runs a `.pkg` installer that needs `sudo`. Use `brew install openjdk@17` (a formula), then set `JAVA_HOME=$(brew --prefix openjdk@17)`.
>
> **AppClaw:** install the **scoped** `@appclaw/cli` (2.x). The old unscoped `appclaw` package is deprecated and fails with `ETARGET / No matching version found for df-vision@1.1.79`.

3 🤖 Android SDK packages

Skip the `sdkmanager` lines if you install these through Android Studio's SDK Manager — but **build-tools 34.0.0 is required either way**: Appium needs `aapt2` from it to parse the APK.

```
# macOS path shown; Linux: $HOME/Android/Sdk
export ANDROID_HOME=$HOME/Library/Android/sdk
mkdir -p "$ANDROID_HOME/cmdline-tools"
# download the command-line tools from developer.android.com/studio#command-line-tools-only
# unzip so the binaries live at: $ANDROID_HOME/cmdline-tools/latest/bin/

SDKM="$ANDROID_HOME/cmdline-tools/latest/bin/sdkmanager"
# Apple Silicon → arm64-v8a   ·   Intel / Linux → x86_64
yes | "$SDKM" "platform-tools" "emulator" "platforms;android-34" \
              "build-tools;34.0.0" \
              "system-images;android-34;google_apis;arm64-v8a"
```

4 🧭 Environment variables

Add to `~/.zshrc` or `~/.bashrc`, then open a new terminal.

```
export JAVA_HOME=$(brew --prefix openjdk@17)         # or your JDK path
export ANDROID_HOME=$HOME/Library/Android/sdk        # Linux: $HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator
export PATH=$PATH:$ANDROID_HOME/build-tools/34.0.0
export PATH=$PATH:$JAVA_HOME/bin
```

5 🧠 The local model

~4.9 GB — do this on good Wi-Fi, not in the room.

```
ollama serve &            # background server on :11434
ollama pull llama3.1      # ~4.9 GB — reasoning, healing and analysis all use it
ollama list               # must show llama3.1
```

6 📦 Clone, install, and get the demo app

```
git clone https://github.com/QA-Olympians-Academy/droid-detective.git
cd droid-detective
pnpm install                        # uses the pinned pnpm@10.28.0
pnpm run appium:install-driver      # idempotent

# the demo APK is git-ignored — download it (WebdriverIO native demo app):
curl -L -o apps/demo.apk \
  "$(curl -s https://api.github.com/repos/webdriverio/native-demo-app/releases/latest | grep -o
'https://[^"]*\.apk')"
unzip -l apps/demo.apk >/dev/null && echo "✓ apps/demo.apk ready"
```

7 🔧 Configure `.env`

```
cp .env.example .env
```

Confirm it contains the defaults — no API key anywhere:

```
LLM_PROVIDER=ollama
LLM_API_KEY=ollama
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=llama3.1
PLATFORM=android
DEVICE_UDID=emulator-5554
APP_PATH=apps/demo.apk
```

8 ✅ Pre-flight smoke test — the gate

Run this end to end and **screenshot the result**. This is the state you should arrive in.

```
# 0. create the AVD — once. Skip if `emulator -list-avds` already shows workshop_avd
#    (or reuse any AVD it lists — substitute its name below).
#    Image must match §3: Apple Silicon → arm64-v8a, Intel/Linux → x86_64.
#    Android Studio alternative: Device Manager → new Pixel 6 / API 34 named workshop_avd.
echo "no" | "$ANDROID_HOME/cmdline-tools/latest/bin/avdmanager" create avd \
  -n workshop_avd -k "system-images;android-34;google_apis;arm64-v8a" -d pixel_6 --force

# 1. boot the emulator (windowed — you will watch the app being driven all day)
emulator -avd workshop_avd -no-audio -no-boot-anim &
adb wait-for-device
until [ "$(adb shell getprop sys.boot_completed 2>/dev/null | tr -d '\r')" = "1" ]; do sleep 3;
done
adb devices                         # expect: emulator-5554  device

# 2. local model responds
curl -s http://localhost:11434/api/tags >/dev/null && echo "✓ ollama up"

# 3. the WDIO suite runs and the app installs
pnpm test
```

✅ **Green condition** — `adb devices` shows `emulator-5554  device`, Ollama is up, and `pnpm test` **starts, installs the app, and runs specs**.

Some assertions may fail — that is fine and expected. The point is that the stack works end to end.

9 ⚡ Shortcut — one-shot check

Once §1–§7 are done, save this as `setup.sh` in the repo root and run it — it verifies everything and fills in whatever is missing.

```
#!/usr/bin/env bash
set -euo pipefail
export ANDROID_HOME=${ANDROID_HOME:-$HOME/Library/Android/sdk}
export JAVA_HOME=${JAVA_HOME:-$(brew --prefix openjdk@17 2>/dev/null || echo /usr/lib/jvm/java-17)}
export PATH="$JAVA_HOME/bin:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator:$ANDROID_HOME/build-tools/34.0.0:$PATH"

echo "checks:"; node -v; pnpm -v; java -version 2>&1 | head -1; adb --version | head -1
appium --version; ollama --version
appclaw --version 2>/dev/null || npm i -g @appclaw/cli

ollama list | grep -q llama3.1 || ollama pull llama3.1
[ -f apps/demo.apk ] || curl -L -o apps/demo.apk \
  "$(curl -s https://api.github.com/repos/webdriverio/native-demo-app/releases/latest | grep -o 'https://[^"]*\.apk')"
pnpm install
pnpm run appium:install-driver
echo "✅ prerequisites satisfied — boot the emulator and run 'pnpm test'"
```

10 If something breaks

SYMPTOM	FIX
`Could not find 'aapt2'`	`sdkmanager "build-tools;34.0.0"` — Appium needs it to parse the APK
`Could not find a driver for 'UiAutomator2'`	`export APPIUM_HOME=$HOME/.appium`, then re-run `appium driver install uiautomator2`
`driver 'uiautomator2' is already installed`	Harmless — the install step is idempotent
`Unknown AVD name [workshop_avd]`	The AVD was never created — run §8 step 0, or boot one from `emulator -list-avds`
`adb: no devices/emulators found`	Emulator not booted (§8); or `adb kill-server && adb start-server`
Emulator is dead-slow	No acceleration — Apple Silicon needs the `arm64-v8a` image; Intel/Linux need HAXM/KVM enabled
`npm i -g appclaw` **fails with** `ETARGET … df-vision`	Install the scoped package instead: `npm i -g @appclaw/cli`
Ollama hangs or is very slow	First load into RAM — run `ollama run llama3.1` once beforehand and close other apps
AppClaw: `model 'llama3.2' not found`	You launched it outside the repo root, so `.env` was not loaded — run from the repo root

🎒 On the day, bring

Your laptop with the checklist verified and the screenshot from §8 · your charger · a GitHub login you can actually sign into. If you get stuck on any step, send the failing command and its full output ahead of the session — that is far quicker than debugging it in the room.